

SkyCL: Swift Continuous Learning with Kinship-Awareness for Multi-Drone Video Analytics under Drastic Drift

Yuanzheng Tan
Shenzhen International
Graduate School, Tsinghua
University
Shenzhen, China
Pengcheng Laboratory
Shenzhen, China

Qing Li*
Pengcheng Laboratory
Shenzhen, China
liq@pcl.ac.cn

Jiaqi Cui
Shenzhen International
Graduate School, Tsinghua
University
Shenzhen, China

Junkun Peng
Shenzhen International
Graduate School, Tsinghua
University
Shenzhen, China

Gareth Tyson
Hong Kong University of
Science and Technology
(GZ)
Guangzhou, China

Zhenhui Yuan
Northumbria University
Newcastle upon Tyne, UK

Tingting Yang
Pengcheng Laboratory
Shenzhen, China

Yong Jiang*
Shenzhen International
Graduate School, Tsinghua
University
Shenzhen, China
Pengcheng Laboratory
Shenzhen, China

Abstract

Drones are increasingly used for on-device video analytics, yet their limited resources can only support lightweight DNNs. To enhance on-device models, continuous learning has been proposed that adapts DNNs to real-time videos by model reuse or retraining. However, existing methods struggle with low accuracy due to drastic drift in agile drones' videos, which leads to outdated retraining, false alarm in drift detection, and intricate performance kinship within the historical model zoo. In this paper, we propose SkyCL, a swift continuous learning system to enhance multi-drone video analytics under drastic drift. SkyCL achieves swift on-device adaptation by dual-level drift detection and distributed retraining scheduling across multiple edge devices. Moreover, SkyCL leverages model kinship within the model zoo to discover a hidden path of models' improvement for potential optimal reuse. We evaluate SkyCL in both simulated and real drone systems, with results showing a 14.39% accuracy gain and a $2.41\times$ speedup over baselines.

CCS Concepts

• **Networks** → **Cloud computing**; • **Computer systems organization** → **Embedded and cyber-physical systems**.

Keywords

Continuous learning, Edge computing, Multi-drone video analytics

ACM Reference Format:

Yuanzheng Tan, Qing Li, Jiaqi Cui, Junkun Peng, Gareth Tyson, Zhenhui Yuan, Tingting Yang, and Yong Jiang. 2026. SkyCL: Swift Continuous Learning with Kinship-Awareness for Multi-Drone Video Analytics under Drastic

* Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792131>

Drift. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3774904.3792131>

1 Introduction

Networked drones become popular for various applications like surveillance and transportation [20, 27, 33]. The requisite video analytics of these uses is usually performed onboard the drones to provide real-time results and comply with privacy constraints [5]. However, resource-limited drones require using pruned or compressed models, which exhibit reduced accuracy under data drift of drones due to their limited scene generalization [16].

Recently, continuous learning (CL) demonstrates the potential to enhance on-device lightweight models under data drift by utilizing real-time data web-transmitted from end devices, including two stages: drift detection and model adaptation. In the adaptation stage, typical solutions retrain models on powerful edge devices using real-time videos from end devices, then distribute updated models to end devices via the web [4, 16]. Though promising, retraining DNNs is limited by its time-consuming nature. Abrupt drift in videos triggers accuracy drops until retraining completes.

To avoid retraining latency, model reuse has been integrated into the adaptation of CL, which reuses models from a collection of historical models and expands the collection by retraining [12, 17, 32]. However, existing solutions suffer from severely low accuracy on drones due to more drastic drift caused by drones' wider field-of-views and higher mobility. Drastic drift in drone-view videos leads to obsolete retraining and false drift alerts, which also demands constant drift detection on end devices that strains their limited resources. In addition, the collection of historical models is expanded more frequently under rapid data drift, creating complex performance inheritance relationships that ultimately lead to the reuse of inferior models. We refer to this relationship as *model kinship*. Our later analysis in §2 conducts an in-depth analysis of the challenges that drones bring to drift detection and model adaption in CL.

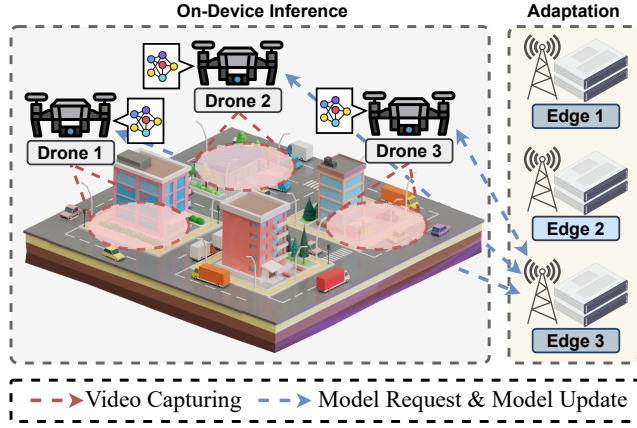


Figure 1: The working scenario of SkyCL.

Our goal: In this paper, we present SkyCL, a swift continuous learning system to enhance multi-drone video analytics under drastic data drift. As in Figure 1, drones analyze captured videos with on-device models. Upon drift in a drone’s video is detected, the update request and the video of this drone are offloaded to an edge device, which reuses a historical model from a model zoo that includes historical models and selectively retrains it on the offloaded video. The reused model and retrained model are successively distributed to the drone for continuous high precision.

Technical challenges: To achieve above goals, following challenges need to be addressed: i) How to achieve low false-alarm drift detection with low computational load in agile drones’ vision? ii) How to utilize model kinship to select the implicit optimal model? iii) How to choose a suitable edge device for rapid retraining?

Solution: To tackle these challenges, we design three key components for SkyCL: a Request Trigger, a Kinship-Aware Selector, and an Offloading Scheduler. i) The *Request Trigger* on drones that precisely detects drift and requests adaptation by analyzing variations in both pixel features and content semantics. Moreover, our CUBIC Dynamic Skipping strategy can reduce computational costs while maintaining accurate detection. ii) The *Kinship-Aware Selector* on edge devices that select the optimal model for reuse. It first uses visual similarity to select an initially suitable model in the model zoo, which serves as the foundation for constructing a model kinship graph. By discovering the hidden connections in this graph, the selector can identify the selected model’s performance improvement path during its retraining, thereby reusing the potential optimal model. iii) The *Offloading Scheduler* on a selected edge (called main edge) that adaptively selects a suitable edge to respond to drones’ requests and allocates resources of this edge for retraining based on computing and network resources of all edges.

We evaluate SkyCL using both simulation and a real-world multi-drone system. Results show that SkyCL achieves 93.21% accuracy, which is 14.39% higher than that of baselines. Moreover, SkyCL is 2.41× faster than baselines in achieving comparable performance.

Contributions: Our key contributions are as follows:

- We propose a swift continuous learning system to address drastic data drift from rapidly-flying drones, enabling accurate multi-drone on-device video analytics.

- We design a Request Trigger to efficiently detect drift and request adaptation with hierarchical drone-vision perception and dynamic skip-frame execution to alleviate load.
- We design a Kinship-Aware Selector to construct the model kinship graph and discover the implicit optimization paths in retraining, thereby reusing the optimal model.
- We design an Offloading Scheduler to schedule retraining offloading and resource allocation according to computing and network resources of distributed edge devices.

2 Motivational Studies

2.1 Experimental Setup

Data. We use the UAVDT video dataset [7] for evaluation, which contains the dynamic content drift from the drone’s view.

Baselines. General continuous learning typically employs an adaptation window to store last video frames for model adaptation. We implement four general methods as baselines [4, 12, 32]:

- ① **Model Retraining (MR)** [4]. Upon detecting drift, MR re-trains the model using frames from the window to update it.
- ② **Content-based Model Selection (MS-C)** [17]. Upon detecting drift, MS-C selects a model trained with the video that has the greatest visual similarity to video frames in the window.
- ③ **Prediction-based Model Selection (MS-P)** [12]. Upon detecting drift, MS-P selects the optimal model by using an additional estimator to predict each model’s accuracy on frames in window.
- ④ **Retraining + Selection (RS)**. This combines retraining and reuse from above to expand the model zoo. The variant incorporating MS-P is termed **RS-P**, which needs to be finetuned due to the expanding model zoo. Conversely, the method with MS-C is called **RS-C**, which uses a static selector for similarity comparison.

2.2 Overall Performance of CL on Drone Videos

Accuracy. Figure 2a shows the accuracy of baselines on flying drone videos, along with the variation of accuracy during a time series. Overall, the performance of all methods is inadequate. The best method (RS-C) has an accuracy of 75.4%, significantly lower than heavyweight DNNs, e.g., Faster-RCNN [28] with 91.2%. Moreover, accuracy often drops abruptly, reflecting the inadequacy of these methods in responding to rapid drift in drone videos.

Retraining Latency. Figure 2b illustrates the retraining latency under varying frame quantities and scene complexity levels. Notably, retraining entails a non-trivial computational overhead even with a small number of frames, resulting in inadequate responsiveness to abrupt drift in video content and limiting real-time adaptation.

Insight #1: *The performance of general methods is inadequate due to their failure to respond to the rapid drift in drone videos, which is primarily affected by the extensive time required for model retraining.*

2.3 Fundamental Limitations in General CL

Continuous learning with model reuse is theoretically capable of enhancing lightweight models against rapid drift. To prove this, we manually detect drift and reuse ideal models from the zoo, which achieve a 92.8% accuracy (Ideal in Fig. 2a). However, results in Figure 2a deliver unsatisfactory performance of other general methods. Our next step is to explore the fundamental limitations for this.

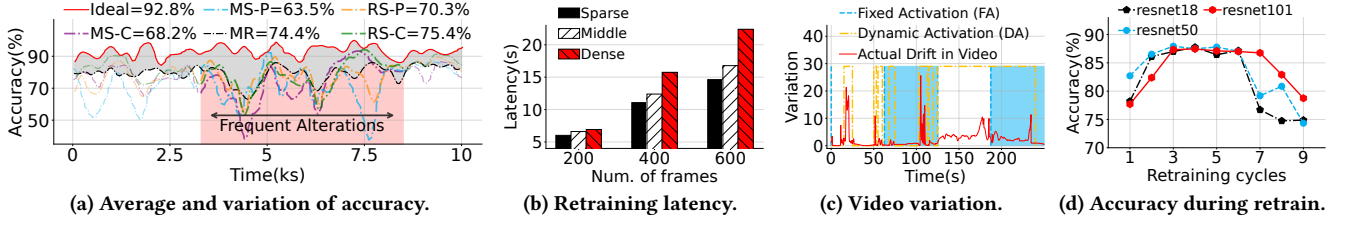


Figure 2: Overall performance of general continuous learning and their limitations on UAVDT dataset.

Inflexible Activation. One limitation is that general methods cannot respond to drastic drift in drone videos due to their inflexible activation. Figure 2c presents the variation in drone videos. Some previous approaches employ periodic model adaptation, which often miss the intra-period drift, triggering rapid model obsolescence (FA in 2c). Advanced methods detect drift by analyzing pixel-level feature [2, 6, 11]. However, flight dynamics like altitude and speed shifts cause complex, jittery drone video drift, leading to frequent false alarm detection and ongoing resource drain (DA in 2c).

Insight #2: *On-device analytics performance might benefit from continuous learning, but only in cases where the adaptation is efficiently activated according to the dynamic drift of drone-view videos.*

Suboptimal Reuse. Another limitation is the suboptimal reuse from the model zoo, as evidenced by the gap between ideal reuse and other reuse methods in 2a. The root cause of this is the rapidly expanding model zoo, triggered by drastic drift in drone video, which undermines accurate model capability assessment. Specifically, a model in the rapidly expanding model zoo is retrained for multiple cycles. Due to DNN’s inherent memorization capacity, retrained models can partially inherit scenes learned by the original model, which is referred to as *model kinship*. However, content-based methods solely rely on a similarity comparison between latest window frames and each model’s most recent training video for reuse. To highlight this conflict, we continuously train a model for nine cycles. In each cycle, the model is trained on a new video, then tested on the original video from the first cycle. Figure 2d reveals that models from later cycles outperform the first-cycle model even on its native training video, suggesting that a kinship sub-model can surpass its original even with less data similarity due to the implicit inheritance from the original data.

Insight #3: *It is also crucial to utilize model kinship in the model zoo to discover potential optimal models for accurate model reuse.*

2.4 Summary and Motivation

Continuous learning’s performance is significantly affected by its activation time and the model kinship, and retraining time also influences its response speed. Based on our insights, we designed SkyCL as follows: i) We schedule retraining across distributed edge devices to speed up its completion. ii) We efficiently detect drift in videos to dynamically activate the adaptation. iii) We perceive the model kinship in the model zoo to reuse the true optimal model.

3 SkyCL Overview

Goal. To enhance multi-drone video analytics under drastic drift, we aim to holistically optimize CL by integrating efficient drift detection, retraining scheduling, and kinship-based model reuse.

Deployment Scenario. Figure 1 shows the working scenario of SkyCL, which can analyze the drone-view videos with consistently high performance. It consists of the following hardware:

① A group of **Drones** that can capture video and run lightweight models for on-device analytics. Drones can also wirelessly connect to edge devices to request model adaptation.

② A group of **Edge Devices** that are distributed and can run heavyweight models. In SkyCL, an edge is selected as **Main Edge**, which collects status of all devices to schedule task offloading.

Operational Flow. The operational flow of SkyCL is illustrated in Figure 3, which consists of following processes:

① **Model Inference & Request** executes on the drone side (§4.1). Each drone uses a lightweight student model to analyze its real-time captured video. Meanwhile, the *Request Trigger* dynamically monitors video drift by dual-level drift detection. Once drift occurs, an update request and selected frames of current video are offloaded to an edge device designated by the main edge.

② **Model Adaptation** takes place on the edge devices (§4.2). Based on the schedule in ③, drones offload their update requests and selected frames to the designated edge, which adapt student models for higher accuracy by reuse and retraining from a historical model zoo. Specifically, a historical model in the model zoo stores a key-value pair: a key of the visual feature from its most recent training video, and a value holding its latest weights. For model reuse, a *Kinship-Aware Selector* first selects a historical model from the model zoo, assisted by visual lookup key and model kinship of the model zoo. If this model underperforms, it is then retrained and added to the model zoo.

③ **Offloading Scheduling** occurs on the main edge (§4.3). It determines which edge the drones should connect to and how many resources should be allocated to retraining, which enables efficient resource utilization across the distributed edges. The scheduler inputs all edges’ computing and network resources, then assigns an edge for adaptation offloading and the retraining configuration, i.e., retraining hyperparameters. Notice, the scheduling is asynchronous with the inference on drones and adaptation on edges.

4 Detailed Design of SkyCL

4.1 Request Trigger

Goal. According to **Insight #2** in §2, our goal is to efficiently activate adaptation to timely respond to dynamic drone-view video drift while minimizing the resource consumption of drift detection.

Design. To achieve this goal, we design the *Request Trigger*, which can efficiently detect drift in videos for timely adaptation request. The workflow of the Request Trigger is as follows:

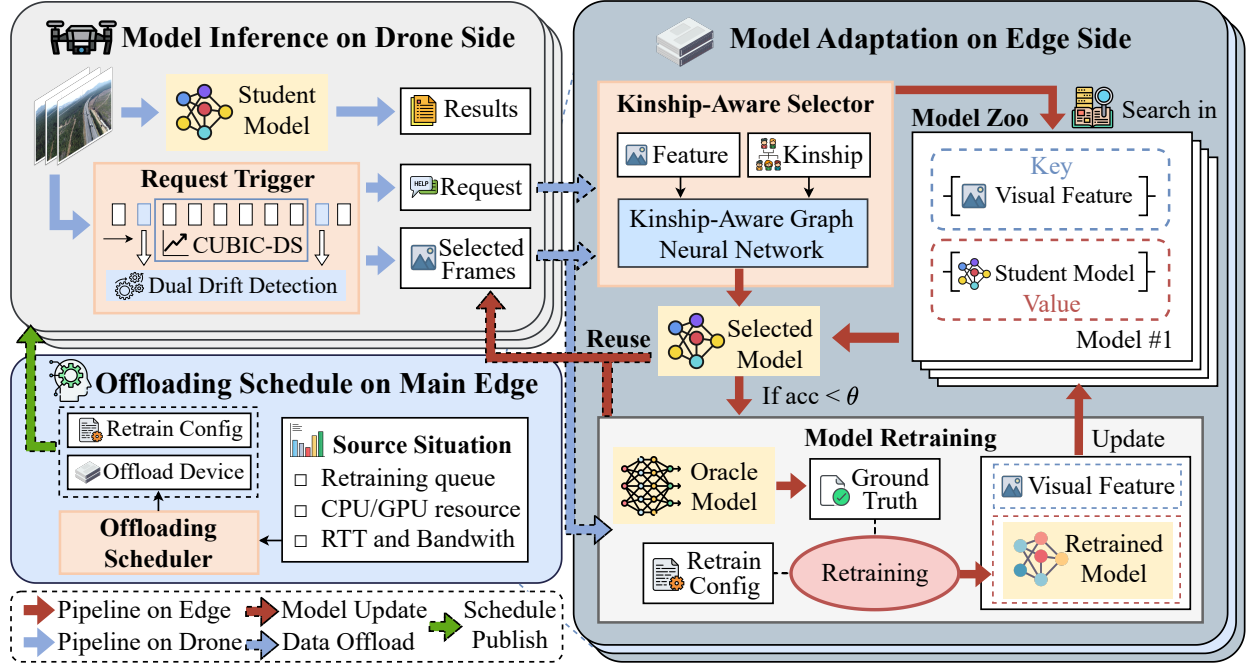


Figure 3: Operational flow of SkyCL in a nutshell.

① **Pixel-Level Change Detection.** Each drone continuously streams video to its local Request Trigger, which first computes a pixel-level change score between two frames to detect outliers of change magnitude. If the score is inlying, it proceeds to Step ② to skip frames. Otherwise, it advances to Step ③ to confirm genuine drift via semantic-level analysis for false alarms avoiding.

② **Dynamic Frame Skipping.** To reduce overhead, the Request Trigger maintains an adaptive window that skips frames when change scores are inlying. Upon inlying detection, the window size is increased (see below) and the window is cleared. New frames are buffered until the window is full, then the next incoming frame and the last saved frame are sent to Step ① for change detection, before that incoming frame is saved.

③ **Semantic-Level Similarity Analysis.** If the change score in Step ① is outlying, the Request Trigger performs semantic-level analysis on the two frames and then determines data drift by combining pixel variation and semantic similarity. If no drift is detected, it retreats to Step ②. Otherwise, it sends frames in the window to the edge for model adaptation. After that, the window is cleared and the window size is decreased, then enters Step ②.

Dual-level Drift Detection. In Step ① and Step ③, we introduce a dual-level drift detection that integrates pixel change and semantic similarity analysis to reduce false alarms. For pixel-level change detection in ①, we use optical flow to quantify inter-frame motion [11, 22], which inputs two frames and provides their optical flow value opt_{cur} . Then we compute the Z-Score of opt_{cur} with historical values, defining this measure as the pixel change score cs :

$$cs = |(opt_{cur} - \mu_{opt}) / \sigma_{opt}|, \quad (1)$$

where μ_{opt}, σ_{opt} is the mean and standard deviation of historical optical flow values. The $cs > 3$ is considered an outlier, prompting further analysis in Step ③ to confirm actual data drift.

For semantic similarity analysis in ③, we use a text-aligned visual encoder [36] to extract semantic feature f_{cur}^v from the current frame. Then, using the similarity of semantic feature and change score, we jointly obtain a drift score ds , defined as:

$$ds = cs[1 - \cos(f_{cur}^v, f_{last}^v)], \quad (2)$$

where $\cos(\cdot, \cdot)$ represents cosine similarity. If ds exceeds threshold α , data drift is confirmed, triggering a model adaptation request.

CUBIC Dynamic Skipping. The Request Trigger skips frames by adjusting the adaptation window size in ② and in ③ for efficient detection. Building upon the CUBIC algorithm for congestion control [9], we propose CUBIC Dynamic Skipping (CUBIC-DS), which adaptively adjusts the window size through drift severity awareness. Specifically, the adjusting of window size is divided into two adjustments: cubic increase and multiplicative decrease. When the pixel-level change is slight (Step ②), the window size is increased as follows to skip redundant frames in the stable scene:

$$w_t = \frac{C}{cs} \left(t - \sqrt[3]{\frac{w_{max} \times \beta}{C}} \right) + w_{max}, \quad (3)$$

where t is the times of pixel-level change detections since the last window decrease, w_t is the size of the t -th window, w_{max} is the max window size before the last window decrease, C, β are respectively the increase and decrease factors. When the drift is confirmed (Step ③), the window size is decreased to finely detect drift in further dynamic segments. The decrease in window size is as follows:

$$w_t = \frac{\beta \times w_{max}}{ds}. \quad (4)$$

By dynamically adjusting the window size, the Request Trigger can skip redundant frames and detect drift efficiently.

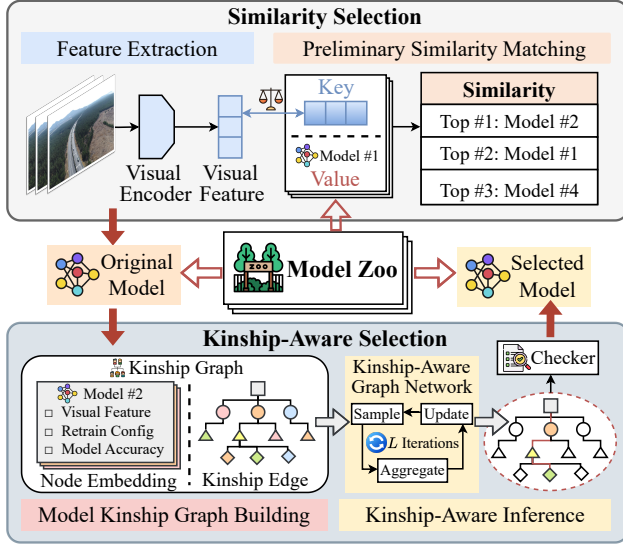


Figure 4: Kinship-Aware Model Selection.

4.2 Kinship-Aware Selector

Goal. According to **Insight #3** in §2, our goal is to utilize model kinship to discover potential optimal models for accurate reuse.

Design. To achieve this goal, we design a Kinship-Aware Selector for optimal model reuse by its two-stage selection, with the workflow detailed in Figure 4 as follows:

① **Similarity Selection.** The Kinship-Aware Selector first selects an original model by comparing visual similarity, serving as a basis for kinship exploration. It extracts a visual feature from the current scene video via a text-aligned visual encoder [36], then retrieves the model trained on the most similar scene by matching against all keys in the zoo. While typical content-based selection treats this model as optimal, the selector uses it as an origin to seek better alternatives through kinship analysis.

② **Kinship-Aware Selection.** The Kinship-Aware Selector further uses model kinship to identify a potentially superior model. It first constructs a directed acyclic kinship graph with the origin as the root, in which nodes represent retraining attributes and edges represent parent-child retraining relationships. The Kinship-Aware Graph Network processes this graph to retain only edges where child models outperform the origin on the origin’s training data. Finally, the performance of the models with retained edges is checked on the current scene video to select the truly optimal one.

Kinship-Aware Graph Network. To harness kinship for optimal reuse, we design a Kinship-Aware Graph Network (KAGN) that identifies high-accuracy retraining paths by preserving performance-gaining edges in kinship. The kinship graph is defined as $\mathcal{K} = (V_{\mathcal{K}}, E_{\mathcal{K}})$, where $V_{\mathcal{K}} \in \mathbb{R}^{n \times 3d}$ is the node matrix of n models in the graph, and $E_{\mathcal{K}} \in \{0, 1\}^{n \times n}$ is the adjacency matrix that describes the retraining kinship. The v -th node feature $h_v \in \mathbb{R}^{3d}$ comprises three d -D embeddings of a model’s retraining attributes: training video features, retraining configuration, and training accuracy.

To handle the kinship graph’s dynamic structure, we develop KAGN upon the GraphSAGE [10], which executes L iterations of sampling, aggregation, and update. During the k -th iteration, KAGN traverses each node, samples s_k neighbors from its neighborhood,

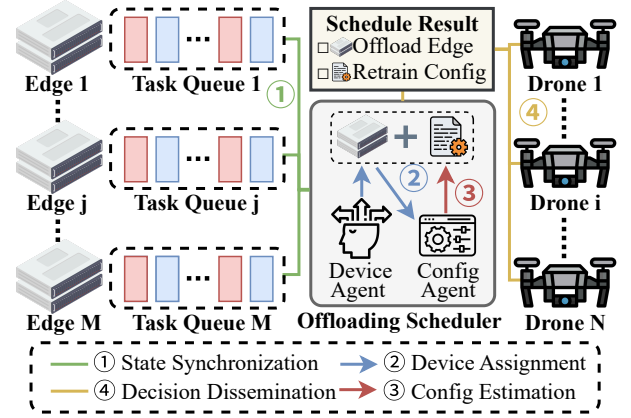


Figure 5: Offloading Schedule.

and constructs a neighborhood subgraph, defined as:

$$\tilde{\mathcal{N}}(v) = \text{Sample}(\mathcal{N}(v), s_k), \quad (5)$$

where $v \in V_{\mathcal{K}}$ is a node in the graph, $\mathcal{N}(v)$ is the full neighbors set of v . Then, KAGN aggregates neighbor features from the last iteration to form an aggregated representation a_v^k in the k -th iteration:

$$a_v^k = \sigma \left(W_{\text{agg}}^k \cdot \text{LSTM} \left(\{h_u^{k-1} \mid u \in \tilde{\mathcal{N}}(v)\} \right) \right), \quad (6)$$

where W_{agg}^k is learnable parameters in the k -th iteration, $\sigma(\cdot)$ is the sigmoid function, and LSTM refers to Long Short-Term Memory-based aggregation [8]. Finally, KAGN updates each node’s feature with its intrinsic feature and the aggregated representation:

$$h_v^k = \sigma \left(W_{\text{up}}^k \cdot \text{CONCAT} \left(h_v^{k-1}, a_v^k \right) \right), \quad (7)$$

where W_{up}^k is learnable parameters in the k -th iteration, and CONCAT is the concatenation between features. After L iterations, KAGN estimates the probability of a performance gain between each node:

$$p(u, v) = \sigma \left(W_{\text{pred}} \cdot \text{CONCAT} \left(h_u^L, h_v^L \right) + b \right), \quad (8)$$

where W_{pred}, b are learnable parameters. KAGN is trained with supervised learning to predict retraining paths that yield performance gains, utilizing binary cross-entropy to identify beneficial edges in the kinship graph. Training labels are obtained by veritally evaluation of all sub-models on the original training data.

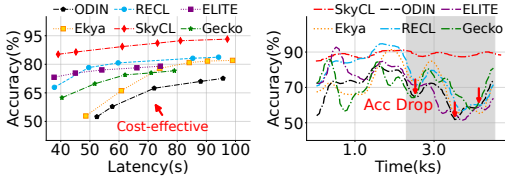
4.3 Offloading Scheduling

Goal. In light of **Insight #1** in §2, our goal is to schedule task offloading and resource allocation to accelerate the adaptation.

Design. We propose an Offloading Scheduler to make this schedule, which consists of two components: a device agent for edge assignment and a config agent for retraining configuration estimation. Since the appearance of adaptation tasks is unknowable, the scheduler assigns a schedule period (e.g., 10 seconds), expressed as P_t . Figure 5 shows the scheduler’s workflow within a P_t :

① **State Synchronization.** At the P_t start, the main edge obtains state s_t of all edges. Then, the scheduler makes a decision in P_t using the device agent (Step ②) and the config agent (Step ③).

② **Device Assignment.** The device agent in the scheduler selects the edge device for offloading, which inputs the state information s_t that encompasses the computing and network resources



(a) Accuracy-Latency. (b) Accuracy variation.

Figure 6: Overall performance comparison.

of all edge devices (see below), and the output a_t^d is the designated edge device for offloading in P_t .

③ **Config Estimation.** After ②, the config agent in the scheduler provides the allocation of computing resources, which inputs state information s_t and designated edge a_t^d to provide a further action a_t^c , including the retraining configuration and the maximum number of retraining tasks for the designated edge device.

④ **Decision Dissemination.** After scheduling, the main edge sends the decision to all drones and edges for tasks offloading. If the number of tasks received by the designated edge exceeds the maximum limit, the next scheduling period will commence earlier.

Scheduler's Training. To obtain the two agents in Step ② and ③ for online decision, we use Reinforcement Learning (RL) to train them. We consider M edges $\mathbb{E} = \{E_1, \dots, E_M\}$, and L retraining configurations $\mathbb{C} = \{C_1, \dots, C_L\}$. One configuration is composed of the number of frozen layers, the batch size, the learning rate, the data sample rate, and the training epochs. The state is given by $s_t = \{Queue_t^j, CPU_t^j, GPU_t^j, Latency_t^j, Bandwidth_t^j\}$, $j = \{1, \dots, M\}$, where $Queue_t^j$ is the length of the unfinished task queue, CPU_t^j and GPU_t^j are the CPU and GPU surplus resources (unit: %), $Latency_t^j$ is the maximum transmission latency between E_j and all drones (unit: ms), $Bandwidth_t^j$ is the available bandwidth of E_j (unit: Kbps). According to s_t , the device agent first performs the action $a_t^d = \{E_j\}$, where E_j is the designated edge for offloading in P_t . Then, the config agent outputs the action $a_t^c = \{C_k, Num\}$ based on s_t and a_t^d , where C_k is the retraining configuration, Num is the maximum number of tasks. The final action $a_t = \{a_t^d, a_t^c\}$ is disseminated to all drones and edge devices. After taking a_t , the Offloading Scheduler receives feedback reward r_t from the system. We expect that a_t could facilitate rapid retraining while maintaining high accuracy, so the reward r_t is designed as:

$$r_t = Acc_t - \beta Time_t^e, \quad (9)$$

where Acc_t is the accuracy of the retrained models, $Time_t^e$ refers to the end-to-end latency of retraining, and β is a scale factor. The policies of the two agents are trained by proximal policy optimization (PPO) [30] to maximize the expected discounted episodic return:

$$\pi^{d*}, \pi^{c*} = \arg \max_{\pi^d, \pi^c} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \right], \quad (10)$$

where π^{d*}, π^{c*} represent the policies of device agent and config agent, γ is the discount factor. By employing RL for interactive learning, the Offloading Scheduler can effectively schedule tasks across edges to speed up retraining while ensuring accurate retraining.

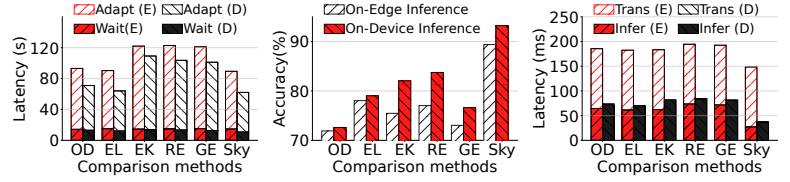


Figure 7: Framework performance evaluation.

5 Dataset Evaluation

Implementation. We deploy SkyCL across two servers, one managing drone tasks and the other handling edge tasks. Each server has 2 Intel E5-2698 v4 CPUs and 8 NVIDIA Tesla V100 GPUs. Distributed operations of model inference and adaptation are simulated via Ray's actor model [23], with GPU resource isolation enabled by NVIDIA MPS [25]. Moreover, drone-edge LTE communication is simulated in ns-3 [1] using real-world flight network traces. We totally run 20 drone processes and 4 edge processes for evaluation. **Data.** Similar to previous studies [4, 12, 13], we collect more than 80 hours of drone-view videos from YouTube for dataset evaluation, covering various environmental, flight, and scene conditions.

5.1 Evaluation of Overall Performance

Baseline Methods. We compare SkyCL with three types of baselines to evaluate the overall performance: i) Retraining-only methods, mainly including *Ekya* [4]. ii) Content-based reuse methods, including *ODIN* [32] and *ELITE* [17]. iii) Prediction-based reuse methods, including *RECL* [12] and *Gecko* [35].

Overall Performance. Figure 6 shows the methods' overall performance on drone videos. Overall, SkyCL has better cost-effectiveness for on-device video analytics than all baselines. Figure 6a shows that SkyCL achieves the highest accuracy of 93.21%, which has an average improvement of 14.39% than baselines. To achieve the same accuracy as SkyCL, the optimal baseline needs to spend more than 2.41 time for retraining. Meanwhile, Figure 6b shows that SkyCL maintains high accuracy during drastic drift in video, while other methods experience multiple accuracy declines. By dynamically reusing suitable models, SkyCL could avoid the time required by retraining and achieve cost-effective continuous learning.

5.2 Evaluation of Framework Performance

We change the frameworks of continuous learning to analyze the impact of the frameworks on accuracy and latency.

Frameworks. We deploy SkyCL and baseline methods in §5.1 on the following frameworks: i) *On-Device Inference (D)* that the inference occurs on end devices like drones, and model adaptation runs on edge devices independently. ii) *On-Edge Inference (E)* that the inference and adaptation are both executed on edge devices in parallel, requiring drone's real-time videos to be completely offloaded from drones to edge devices.

Accuracy. Figure 7b shows accuracy across the two frameworks. SkyCL achieves an average 27.06% higher accuracy than all baseline methods across both frameworks. Due to resource competition from inference workloads, retraining faces stringent computational constraints in on-edge inference framework, leading to significant accuracy degradation in retraining-only methods. Reuse methods

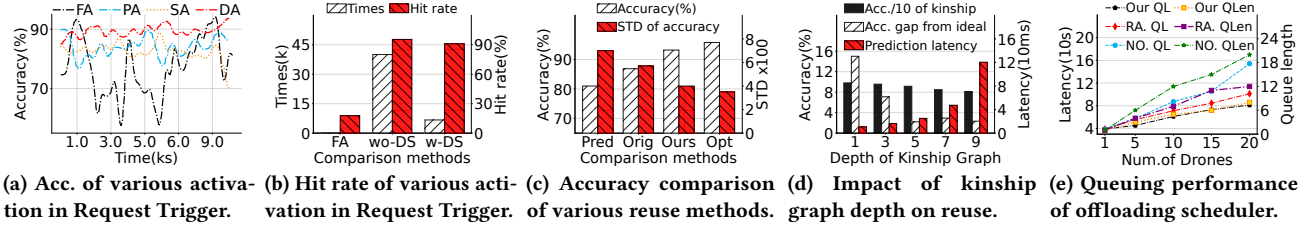


Figure 8: SkyCL's modules performance evaluation.

leverage reuse to mitigate retraining bottlenecks, but their reuse of suboptimal models undermines their accuracy. On the contrary, SkyCL achieves higher accuracy by potential optimal model reuse and selectively retraining, reducing time and resource consumption. **Request Response Latency.** The response latency of adaptation request under resource constraints includes both waiting latency and adaptation latency. Figure 7a shows the response latency of all methods across different frameworks. SkyCL's response latency is the lowest and the waiting latency being $1.21\times$ faster than all the other methods. Moreover, since on-edge inference occupies computational resources of edge devices, the response latency of all methods are increased in on-edge inference frameworks, showing that the on-device inference benefits real-time video analytics.

Model Inference Latency. The on-edge framework's inference latency comprises two components: transmission latency (due to video offloading to the edge) and execution latency, contrasting with on-device inference's exclusive reliance on execution latency. Figure 7c demonstrates that SkyCL achieves the shortest inference latency on both frameworks. Moreover, despite advancements in codec technology, video streaming remains time-consuming under heavy edge resource utilization, while task parallelism on edge slows inference processing. Consequently, each method in on-device inference achieves $3.97\times$ lower latency than on-edge inference, demonstrating clear advantages for real-time analytics.

5.3 Evaluation of Modules Performance

SkyCL's Request Trigger Performance. We compare different model request methods as follows: i) *Fixed Activation (FA)* that periodically starts adaptation at a fixed frequency, which is widely used in general methods. ii) *Pixel Activation (PA)* that only uses pixel-level optical flow to determine drift and request adaptation. iii) *Semantic Activation (SA)* that only uses semantic feature. iv) *Dual Activation (DA)* that leverages dual-level features designed in the Request Trigger. Figure 8a shows the accuracy during the video analytics under different methods. FA's severe performance loss demonstrates significance of drift detection, however, relying solely on pixel or semantic features causes false alarms or missed detection respectively, leading to unstable performance. Our dual-level detection sustains stable accuracy (85-95%) by its precise drift identification. Figure 8b shows that our CUBIC-DS algorithm cuts calculation times by 83.2% while keeping the hit rate of drift detection above 90%, which reflects that our Request Trigger could efficiently detect drift in video and timely request models.

SkyCL's Kinship-Aware Selector performance. Figure 8c examines the performance of various reuse methods, including the prediction-based method (*Pred*), content-based method (*Orig*) and our kinship-aware reuse (*Ours*). *Opt* is the actual optimal model

for current video. Results show that the models we reuse are the closest to ideal optimal models. Rapid model zoo expansion distorts performance estimation of *Pred* due to frequent fine-tuning, while *Orig* can only select suboptimal models due to the complex model kinship. Based on the original model provided by *Orig*, our method explores the performance improvement process in the model kinship, thereby reusing the potential optimal model. Moreover, we test the impact of the depth of the model kinship graph on the reuse performance. Figure 8d shows that deeper kinship graphs are more likely to contain optimal models, but incurring increased inference latency and distorted prediction of performance gains.

SkyCL's Scheduler Performance. Figure 8e illustrates the queue latency (QL) and the queue length (QLen) of different schedulers under varying numbers of drones. The SkyCL system with our Offloading Scheduler shows an increasing advantage in task handling latency as drone numbers grow. In addition, as the number of drones increases, the number of tasks in the waiting queue increases, which leads to an increase in latency. Even so, the latency increase in our Offloading Scheduler is the smallest due to its suitable decision.

6 Real-flight Evaluation

6.1 Experiment Setup

We deploy SkyCL on the real-world testbed for real-flight evaluation. The drones in SkyCL are deployed using FanciSwarm, and the edges are deployed with servers in §5. We run SkyCL for on-device video analytics in a 20 km^2 test area, which encompasses a variety of scenarios, e.g., campuses, urban areas, and parks. The test area is divided into four blocks, each with up to five drones capturing 1080p video at 24 fps. We totally conduct nine experiments across three time periods, including: i) daytime that has plenty of light, ii) dusk that involves the change of light from bright to dark, and iii) night that has only the streetlights to illuminate objects. Each experiment is conducted for 30 minutes.

6.2 Real-flight Evaluation Results

Accuracy. Figure 9a shows the accuracy of SkyCL in three time periods. SkyCL's daytime accuracy reaches 90%, benefiting from optimal scene illumination. At night, accuracy remains high at 88.38% despite scene darkness, because drones can only detect objects that are illuminated by streetlights. Accuracy drops to 83.43% at dusk, the variation of which is visualized in Figure 9b, further highlighting its instability. This is due to numerous pedestrians and vehicles combined with rapid lighting changes at dusk, which causes drastic data drift. Despite such adverse conditions, SkyCL maintains acceptable accuracy due to its efficient reuse and retraining.

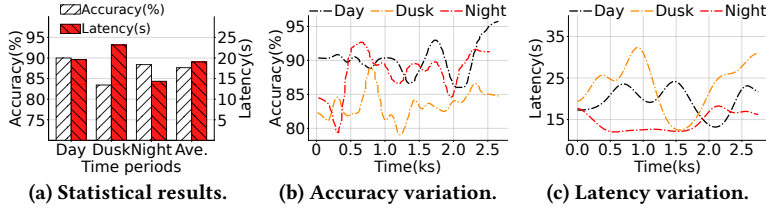


Figure 9: Real-flight evaluation results.

Retraining Latency. Figure 9a shows retraining latency during three different time periods, indicating that the system performs the best at night that the latency stays below 20 seconds, and the fluctuation remains minimal. During the daytime, the latency is at a normal level, fluctuating between 15 and 25 seconds. At dusk, the latency peaks at 45 seconds, accompanied by large fluctuations. These results are closely related to the drift detection in the Request Trigger. During the daytime and night, small changes in light do not trigger new model requests, resulting in a low workload of retraining. On the contrary, due to the intense changes in light during dusk, more requests for new models are triggered, leading to increased waiting and execution latency of the retraining.

Impact of the Number of Drones/Edges. Figure 10 shows all methods' accuracy across varying numbers of drones and edges in real-flight evaluation. SkyCL's accuracy scales with drone fleet size, as more drones enhance performance by adding more reusable models to the model zoo. However, rapid model zoo expansion can undermine the estimators in prediction-based methods (*RECL* and *Gecko*), compromising their performance. In addition, edge device quantity directly influences accuracy by enabling faster adaptation by sufficient computational resources. Despite limited edge devices, SkyCL shows superior efficiency, while other methods suffer from outdated adaptation, leading to performance degradation.

7 Related Work

Recent techniques could deploy lightweight models on end devices [3, 18, 19, 21]. However, a challenge faced by lightweight models is data drift, requiring continuous adaptation to the real-time video stream. Existing adaptation methods rely on two approaches: model retraining [4, 13, 24] and model reuse [12, 17, 29, 32, 35].

Model Retraining methods retrain the model on real-time video stream to address data drift of on-device models, which can retain learned knowledge while incorporating new data [6, 26, 31, 34]. AMS [13] improves the accuracy by retraining models using the dynamically sampled frames in real-time videos from end devices. EOMU [16] also samples representative frames and optimize retraining configurations to enhance video analytics. However, because retraining is time-consuming, these methods struggle to keep pace with rapidly drift in drone-view videos. SkyCL effectively integrates reuse and retraining, enabling the model to adapt to drastic drift.

In addition, a key retraining challenge is resource allocation, necessitating a trade-off between model accuracy and latency to meet real-time analytics demands under limited resources. Ekya [4] addresses this by predicting costs and accuracy under different retraining configurations, effectively balancing accuracy with resource constraints. SkyCL addresses this by distributing retraining tasks across multiple devices, optimizing resource utilization.

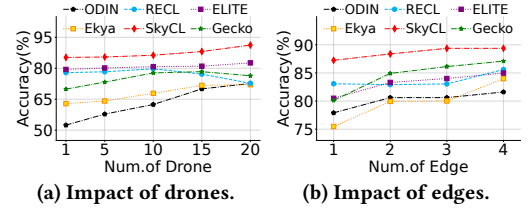


Figure 10: Impact of drones & edges number.

Model Reuse precisely reuses models from the collection of historical models to avoid retraining latency [12, 14, 29, 32]. There are two primary approaches: content-based and prediction-based.

The content-based methods select models by analyzing the content of input videos. ODIN [32] employs an unsupervised algorithm that clusters visual features from videos to select the most appropriate model. ELITE [17] introduces a task-oriented on-device selection mechanism for rapid model retrieval and reuse.

Instead of using video content, prediction-based methods use an additional DNN to predict the performance of historical models and select the best one. DWM [15] predicts the weights of each model. Low-weight models are eliminated to manage sustainable updating. RECL [12] uses a gating network to predict the accuracy of all models on the current video to reduce computational requirements during inference. Gecko [35] ulteriorly employs an accuracy predictor with video content-awareness for suitable model reuse.

However, complex model kinship as the model zoo expands leads to suboptimal reuse due to inaccurate performance estimation. SkyCL uses the kinship-aware selector to discover the hidden performance improvement path during retraining, thereby reusing the potential optimal model.

8 Conclusion

In this paper, we introduced SkyCL, a swift continuous learning system that incorporates model reuse and retraining to enhance multi-drone video analytics under drastic drift. To enhance SkyCL's responsiveness to drift in drone-view videos, we developed a Request Trigger with our dual-level drift detection and CUBIC-DS algorithm to efficiently detect drift in videos. We also designed an Offloading Scheduler for SkyCL to distribute retraining tasks across multiple edge devices to enhance efficiency. To achieve accurate adaptation, we dived into the impact of model kinship on reuse, and proposed a Kinship-Aware Selector to precisely reuse models by discovering hidden performance improvement paths in model zoo. Our comprehensive simulated and real-world validations demonstrate that SkyCL enables swift and resource-efficient on-device video analytics for multi-drone platforms in the web of things.

9 Acknowledgment

This work is supported by Project of the Department of Strategic and Advanced Interdisciplinary Research of Pengcheng Laboratory under Grant. 2025QYA001, the Major Key Project of PCL under grant NO. PCL2025A09, the National Key Research and Development Program of China under grant No. 2022YFB3105000, and the Shenzhen Key Lab of Software Defined Networking under grant No. ZDSYS20140509172959989.

References

- [1] 2024. ns3 - Network Simulator. <https://www.nsnam.org/>.
- [2] Tudor Barbu. 2014. Pedestrian detection and tracking using temporal differencing and HOG features. *Computers & Electrical Engineering* 40, 4 (2014), 1072–1079.
- [3] Eden Belouadah and Adrian Popescu. 2019. IL2M: Class Incremental Learning With Dual Memory. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [4] Romil Bhardwaj, Zhengxu Xia, Ganesh Ananthanarayanan, Junchen Jiang, Yuan-chao Shu, Nikolaos Karianakis, Kevin Hsieh, Paramvir Bahl, and Ion Stoica. 2022. Ekya: Continuous learning of video analytics models on edge compute servers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 119–135.
- [5] Haowei Chen, Liekang Zeng, Xiaoxi Zhang, and Xu Chen. 2022. Adadrone: Quality of navigation based neural adaptive scheduling for edge-assisted drones. In *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 548–558.
- [6] Yaofu Chen, Shuaicheng Niu, Shoukai Xu, Hengjie Song, Yaowei Wang, and Minghui Tan. 2024. Towards Robust and Efficient Cloud-Edge Elastic Model Adaptation via Selective Entropy Distillation. *arXiv preprint arXiv:2402.17316* (2024).
- [7] Dawei Du, Yuankai Qi, Hongyang Yu, Yifan Yang, Kaiwen Duan, Guorong Li, Weigang Zhang, Qingming Huang, and Qi Tian. 2018. The unmanned aerial vehicle benchmark: Object detection and tracking. In *Proceedings of the European conference on computer vision (ECCV)*, 370–386.
- [8] Alex Graves. 2012. Long short-term memory. *Supervised sequence labelling with recurrent neural networks* (2012), 37–45.
- [9] Sangtae Ha, Injong Rhee, and Lisong Xu. 2008. CUBIC: a new TCP-friendly high-speed TCP variant. *ACM SIGOPS operating systems review* 42, 5 (2008), 64–74.
- [10] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [11] Eddy Ilg, Nikolaus Mayer, Tonmoy Saikia, Margret Keuper, Alexey Dosovitskiy, and Thomas Brox. 2017. FlowNet 2.0: Evolution of optical flow estimation with deep networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2462–2470.
- [12] Mehrdad Khani, Ganesh Ananthanarayanan, Kevin Hsieh, Junchen Jiang, Ravi Netravali, Yuanchao Shu, Mohammad Alizadeh, and Victor Bahl. 2023. {RECL}: Responsive {Resource-Efficient} continuous learning for video analytics. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 917–932.
- [13] Mehrdad Khani, Pouya Hamadani, Arash Nasr-Esfahani, and Mohammad Alizadeh. 2021. Real-time video inference on edge devices via adaptive model streaming. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 4572–4582.
- [14] Jeremy Z Kolter and Marcus A Maloof. 2005. Using additive expert ensembles to cope with concept drift. In *Proceedings of the 22nd international conference on Machine learning*, 449–456.
- [15] J Zico Kolter and Marcus A Maloof. 2007. Dynamic weighted majority: An ensemble method for drifting concepts. *The Journal of Machine Learning Research* 8 (2007), 2755–2790.
- [16] Yuxin Kong, Peng Yang, and Yan Cheng. 2023. Edge-assisted on-device model update for video analytics in adverse environments. In *Proceedings of the 31st ACM International Conference on Multimedia*, 9051–9060.
- [17] Haibo Liu, Chen Gong, Zhengzhe Zheng, Shengzhong Liu, and Fan Wu. 2025. Enabling Real-Time Inference in Online Continual Learning via Device-Cloud Collaboration. In *Proceedings of the ACM on Web Conference 2025*, 2043–2052.
- [18] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. 2016. *SSD: Single Shot MultiBox Detector*. Springer International Publishing, 21–37. doi:10.1007/978-3-319-46448-0_2
- [19] Yucheng Low, Joseph Gonzalez, Aapo Kyrola, Danny Bickson, Carlos Guestrin, and Joseph M Hellerstein. 2012. Distributed graphlab: A framework for machine learning in the cloud. *arXiv preprint arXiv:1204.6078* (2012).
- [20] Haoyu Luo, Tianxiang Chen, Xuejun Li, Shuangyin Li, Chong Zhang, Gansen Zhao, and Xiao Liu. 2022. KeepEdge: A knowledge distillation empowered edge intelligence framework for visual assisted positioning in UAV delivery. *IEEE Transactions on Mobile Computing* (2022).
- [21] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. 2020. Themis: Fair and Efficient GPU Cluster Scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 289–304. <https://www.usenix.org/conference/nsdi20/presentation/mahajan>
- [22] Sidi Ahmed Mahmoudi, Michał Kierzyński, Pierre Manneback, and Krzysztof Kurowski. 2014. Real-time motion tracking using optical flow on multiple GPUs. *Bulletin of the Polish Academy of Sciences. Technical Sciences* 62, 1 (2014), 139–150.
- [23] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, 561–577.
- [24] Ravi Teja Mullapudi, Steven Chen, Keyi Zhang, Deva Ramanan, and Kayvon Fatahalian. 2019. Online model distillation for efficient video inference. In *Proceedings of the IEEE/CVF International conference on computer vision*, 3573–3582.
- [25] TESLA NVIDIA. 2019. MULTI-PROCESS SERVICE.(2019). (2019).
- [26] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. 2019. Continual lifelong learning with neural networks: A review. *Neural networks* 113 (2019), 54–71.
- [27] Junkun Peng, Qing Li, Yuanzheng Tan, Dan Zhao, Zhenhui Yuan, Jinhua Chen, Hanling Wang, and Yong Jiang. 2023. SkyNet: Multi-Drone Cooperation for Real-Time Identification and Localization. In *IEEE International Conference on Computer Communications (INFOCOM)*, 1–10.
- [28] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. 2015. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems* 28 (2015).
- [29] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. 2021. Scaling vision with sparse mixture of experts. *Advances in Neural Information Processing Systems* 34 (2021), 8583–8595.
- [30] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [31] Shivangi Srivastava, Maxim Berman, Matthew B Blaschko, and Devis Tuia. 2019. Adaptive compression-based lifelong learning. *arXiv preprint arXiv:1907.09695* (2019).
- [32] Abhijit Suprem, Joy Arulraj, Calton Pu, and Joao Ferreira. 2020. Odin: Automated drift detection and recovery in video analytics. *arXiv preprint arXiv:2009.05440* (2020).
- [33] Yuanzheng Tan, Qing Li, Junkun Peng, Zhenhui Yuan, and Yong Jiang. 2024. Air-CAD: Edge-Assisted Multi-Drone Network for Real-time Crowd Anomaly Detection. In *Proceedings of the ACM on Web Conference 2024*, 2817–2825.
- [34] Liang Wang, Kai Lu, Nan Zhang, Xiaoyang Qu, Jianzong Wang, Jiguang Wan, Guokuan Li, and Jing Xiao. 2023. Shoggoth: towards efficient edge-cloud collaborative real-time video inference via adaptive online learning. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [35] Liang Wang, Xiaoyang Qu, Jianzong Wang, Guokuan Li, Jiguang Wan, Nan Zhang, Song Guo, and Jing Xiao. 2024. Gecko: Resource-efficient and accurate queries in real-time video streams at the edge. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 481–490.
- [36] Beichen Zhang, Pan Zhang, Xiaoyi Dong, Yuhang Zang, and Jiaqi Wang. 2024. Long-CLIP: Unlocking the Long-Text Capability of CLIP. *arXiv preprint arXiv:2403.15378* (2024).